

Atlas Mod Project

[A CaffeinatedAndroid Project](#) about A modular, low-cost humanoid robotics platform for exploring embodied AI, distributed control, and biologically inspired learning.

ABOUT ME:

I have had an interest in technology from a young age, particularly in robots and AI. I spend the majority of my time working on side projects or repairing what I can. Ive previously attained a diploma in games and a certificate in IT when I was 19. I enrolled in mechatronics at QUT however could not keep up with the cost nor the stress at the time due to illness. So I figured I will find my own way by taking on challenging projects that cover a large aspect of robotics, from something as simple as cable making, all the way to building the software ecosystem and learning simulation. After years I stumbled across SAEs Computer Science Degree and it offered many of the topics I wanted to learn more about and skills to fill in the gaps that diy learning often leaves. Since then I have aimed to apply what I learn to my own projects.

INITIAL PROJECT GOALS

This project came from a desire to learn full-stack robot development and a near-lifelong interest in AI and machine learning. The goal has slowly narrowed into building and completing a research platform similar to the older NAO robots at a fraction of the cost, that can be used to test machine embodiment as well as develop and test varying, more compatible AI architectures for social robots. Along with this, the additional goal is to design hardware and a software base that can be used in custom chassis designs and also allow me to expand into building larger machines.

NAO ROBOT



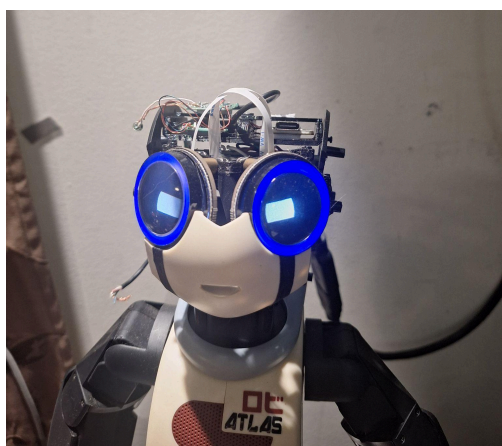
Gen1 Robi



TECHNICAL SPECIFICATIONS

HARDWARE:

Starting with a base chassis from De Agostini, an Italian hobby company who released a diy robot called Robi, designed by Tomotaka Takahashi, a Japanese robotics expert and University of Tokyo associate professor, which were released as part-work build-it-yourself magazines, with a preliminary release in 2012 and then officially in 2013. These robots use Futaba RS308MD (De Agostini UK Ltd., 2015, p. 40) robotic servos that communicate via a single wire asynchronous half-duplex TTL communication. All original hardware except for the serial servos were stripped out, this included the original micro-controllers that drove the servos.



Initial part bashing with random breakouts and damaged audio hardware. Messy but proved to me my initial idea is viable

A question may arise as to why not just design and build a custom chassis. Given I have had prior experience designing a small, quadruped robot, I know how much the servos will cost and what issues I will run into when designing. These are things I would rather address later and the initial focus was to calculate the cost, compute and learn the software side rather than constantly working in CAD and never getting far enough to build a full and refined system. By using broken generation 1 Robis as a base, the servo cost is cut from more than ~\$600 to ~\$250 for 20 robotic servos and a known working chassis. Given a second hand Dynamixel servo (Fig 1, see appendices) can often go for around ~\$40 and cheaper RC hobby servos do not have the accuracy, precision or longevity of robotic servos and often burn out with frequent cycling, this turned out to be a better decision when choosing servos for what was required and allowed me to free up major resources for the project.

With a solid base to work with, I could focus on what mattered which is more powerful computing and new sensors. This involved experimenting with an Arduino, an ESP32 C3 Mini, Raspberry Pi 4 (2019) and a Raspberry Pi Zero 2w (2021) which fit well in the robot's head with sufficient capacity

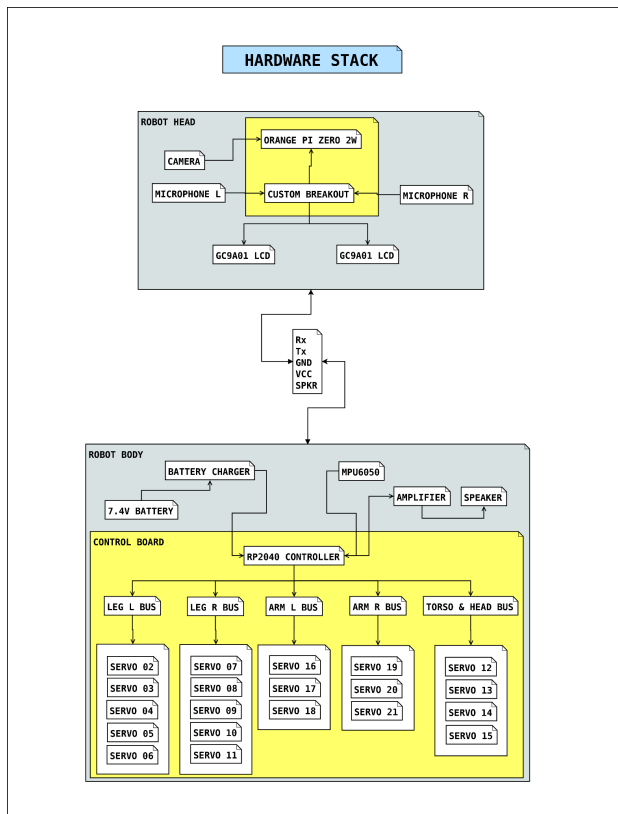
remaining. I used this for initial testing of additional sensors such as i2s mics, cameras and output devices such as the speaker and custom eye LCDs.

One issue that arose was being constrained by the limited ram of the Raspberry Pi zero 2w and I started searching for SBCs that shared the same form factor, ultimately picking the Orange Pi zero 2w with 4GB of ram (Shenzhen Xunlong Software Co., 2023). It performs well, however this is where I encountered an additional layer of complexity in that these devices did not share the same device overlays or device trees or even camera support (though the same connector) that have been curated on the Raspberry Pi platform. Some of this seems to have been addressed in the Zero 3W variant, this will likely be the target device in future as it supports 12GB of RAM in the same form factor and more optimized configs.

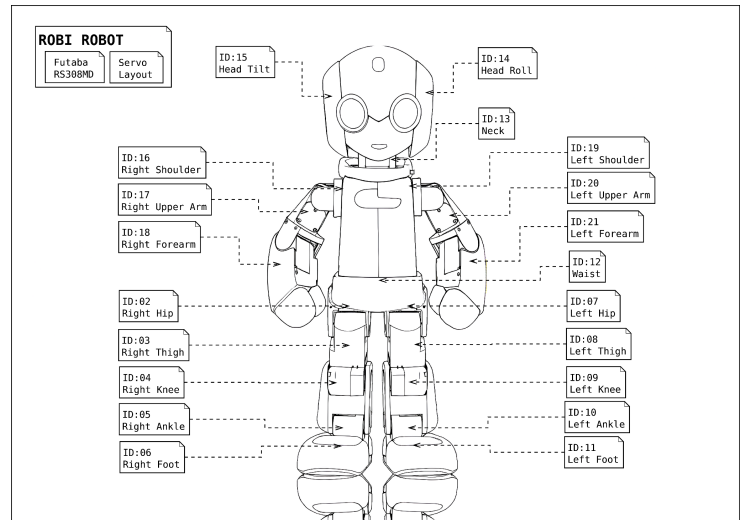
In spite of this I have been able to design a custom breakout board that allows for the displays to connect, while also providing a 6 pin JST SH connection for the main command cable that runs into the body of the robot and connects to a signal splitter for now to distribute signals to the servo chains. This now provides a clean and stable base, allowing focus to be directed at the new micro controller, which will allow each servo chain to be driven independently and prioritize certain limbs such as legs over the arms. This also helps decrease bandwidth usage on the communication bus that can lead to missed signals in testing or inconsistent time of arrival for servo commands which cause erroneous movements.

The current iteration of the custom micro-controller is based around an RP2350B chip, chosen for its large GPIO count and PIO state machines which allow configuring of multiple dedicated serial ports for sensor and servo communications at 115200 Baud (Futaba Corporation, pp.9, 2011), a limitation that could not be overcome with devices such as the ESP32 or Arduino lines given their limited software serial speeds and the size constraints of the robot. The servo speed can be reprogrammed per servo however it is better to work with the recommended speed and has provided a base requirement for me to work with when designing new hardware.

Currently a single serial port is used through an ESP32 breadboard until the new dev board arrives.

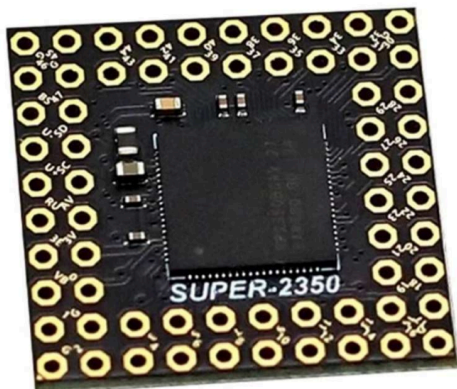


ESP32 Test Setup

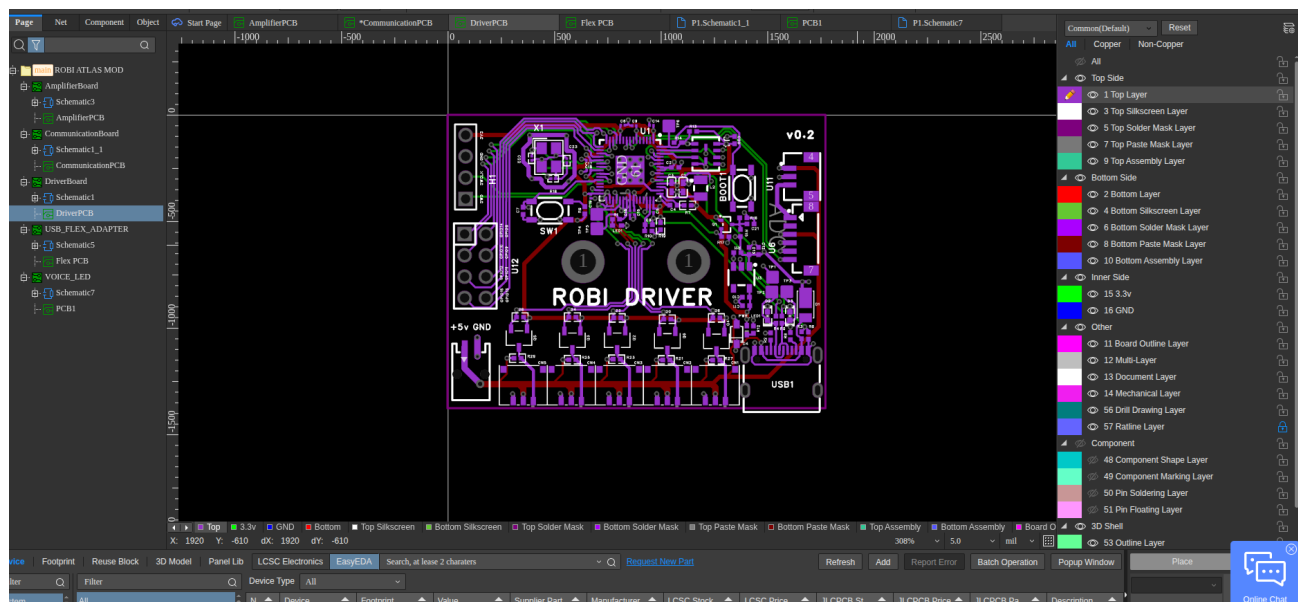


Servo layout, using original IDs

A larger prototype version has been designed using a RP2350 Dev board which has the same chip. This is used for confirming everything works before committing to having the new boards manufactured by JLC PCB as they are much more expensive than the simple breakout designed for the LCDs and communication cable.



RP2350B Bare Dev Board

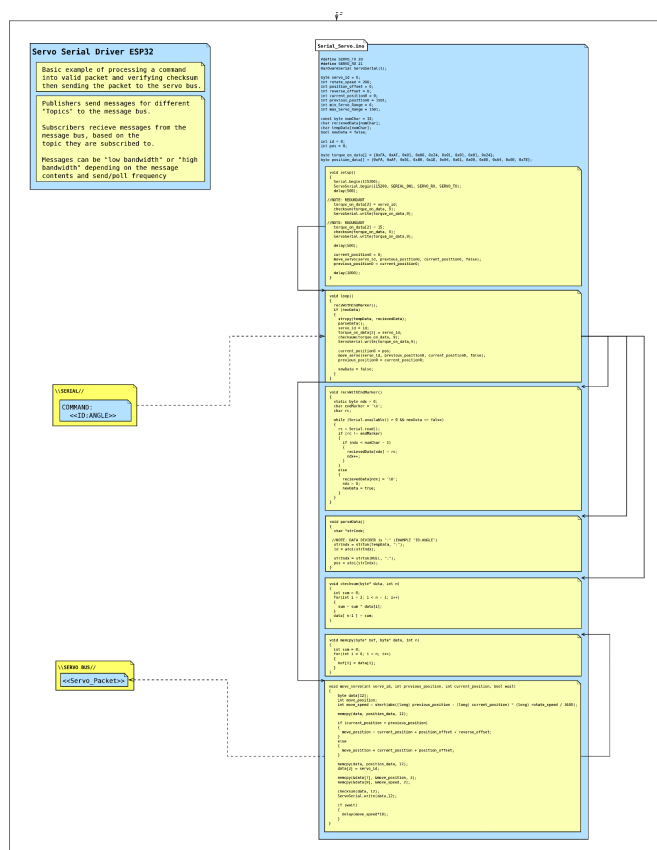


Current RP2350B Microcontroller Draft

SOFTWARE:

The current version uses Dietpi running through tiger VNC and ssh for ease of development.

The initial version of the software called “Robi Mindware” (homage to the old Sony Aibo robot dogs) uses a central message bus with independent queues, this allows messages to be received by subscriber nodes that can independently queue messages such as camera frames, accelerometer data or pose commands to avoid congesting the main message bus.



Communication from SBC (Orange Pi) to Microcontroller

As it stands, for the initial software my goal is to replicate the original “kiosk” like mode of the robot with some extra features such as object tracking and general voice chat using a local pretrained model, this provides a similar starting point to what is seen in current consumer robots and game AI; Often where regular chat-bots have been hooked up alongside generic control software, and lack true integration with the given environment.

An easy to see example of this is in AI skyrim companion mods, where the AI agent is connected to the world through prompts and text interfaces which allow for some interesting behaviour but lacks true integration in the game world that would allow for more emergent behaviours and complex interactions given the context of that games world.

While basic, an initial kiosk mode provides the functioning base to expand upon, and also stands as a base for others to start experimenting with too.

The next step is exploring the creation of custom AI frameworks that could even include experimenting with more complex neurons that simulate biological counterparts that could allow for more complexity or emergent properties.

Connecting to cloud compute at this stage will allow for real-time training of far larger and more complex networks such as large Spiking Neural Networks (SNNs) and Convolutional Neural Networks (CNNs) than what feasibly can be hosted locally, that said with the base software serving as a bridge for sensors and local networks to communicate, this can act more like brain bifurcation rather than tele-operation of a robot body though text.

Currently I have experimented with smaller CNN and SNNs, learning how different types of neural networks function and exploring how base neurons work and how they can be expanded upon. They are only building blocks that can be used to build larger systems and are extremely limited by that. Each test I ran involved tuning neuron parameters to avoid overfitting to data (Grey or boring output) or decay of the network (Failure to propagate signals or gradual garbling of the output reconstructions).

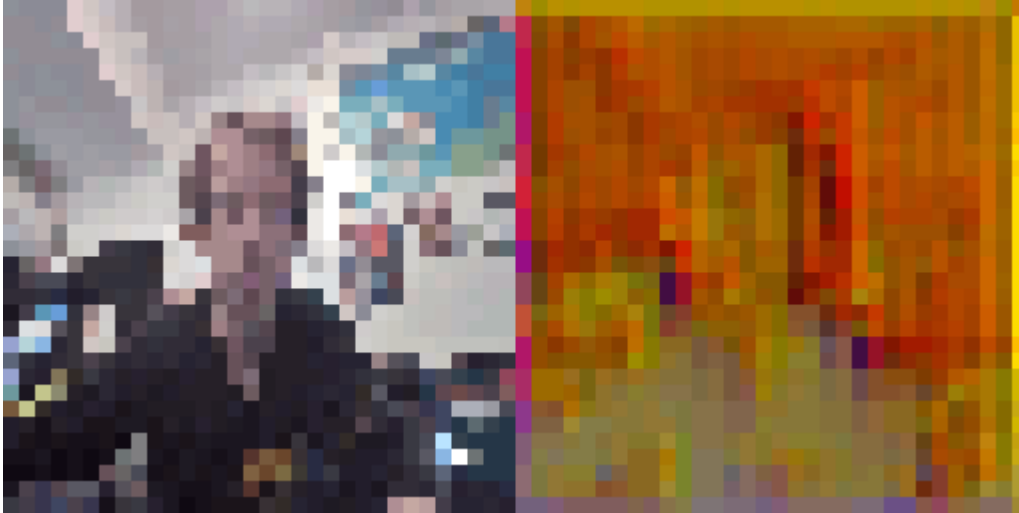
However this is enough to see feature learning such as dark and light regions or identifying outlines and to better develop an understanding of how these systems function on a more foundational level.

Some of the networks include:

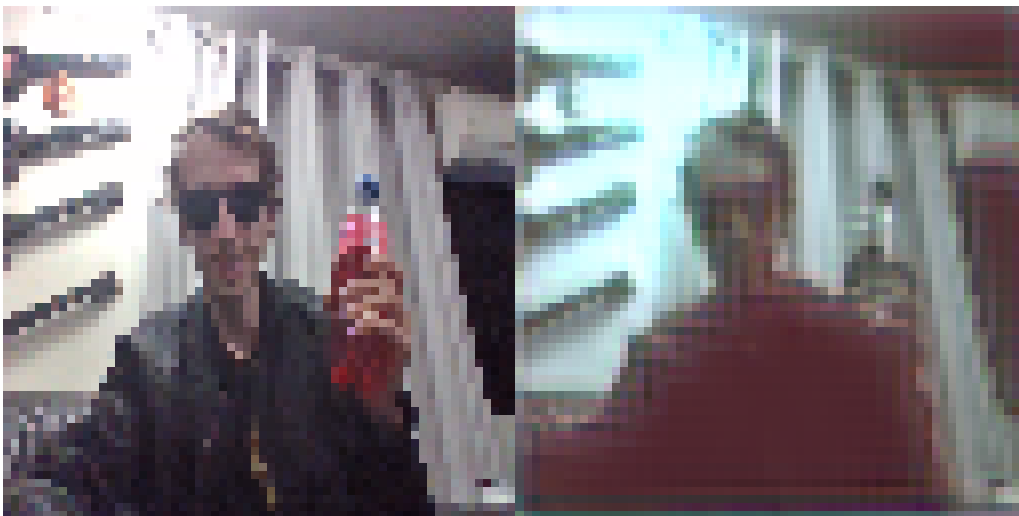
. Color training CNN:



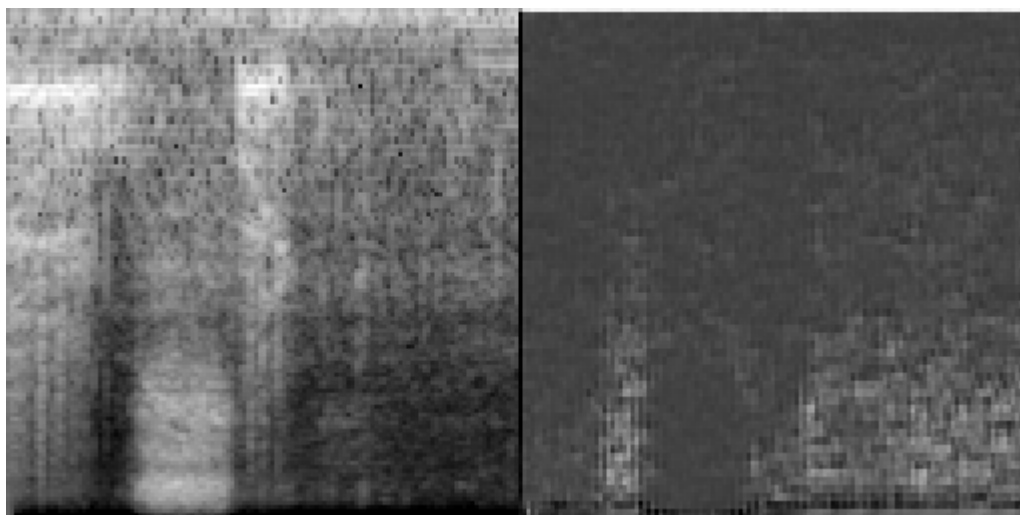
. BIO Color Training SNN:



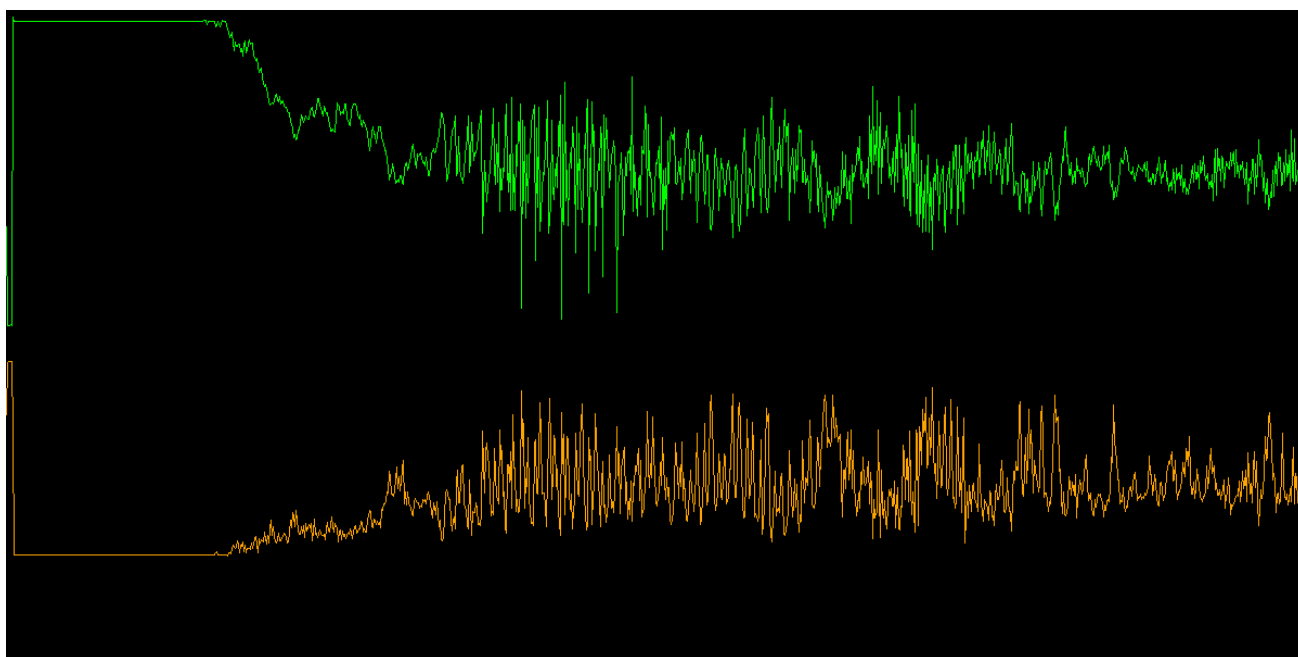
. Color Training Dual Encoder CNN (limited epochs <10)



. SNN Audio (No waveform encoder) (sample says “One”)



SNN(Waveform Encoder) (sample says “one, two, three, four”(High Noise))



The Bio inspired networks rely on an array of variables being dialed in carefully to function, for a simple bio inspired network, there are many more configurations that are possible and complexity grows fast. These networks are however much more chaotic in my testing and require a lot more fine tuning to prevent rapid decay of the network which can appear as failed learning but in reality the networks decays within the first few learning steps. However my initial experiences with them is that

they seem to learn far faster than a standard SNN once dialed in.

```

home ▸ waffles ▸ Robi Mindware Local Dev ▸ Mindware Kernel ▸ Working Demos ▸ Neurons ▸ BioSNN ▸ BioSNN.py
4  import time
5  import psutil
6
7  # ===== GLOBAL CONFIG =====
8  IMG_SIZE      = 64
9  NUM_FILTERS   = 32
10 KERNEL_SIZE   = 3
11 LEARNING_RATE = 0.00015      # Lowered for spiking dynamics
12 EPOCHS        = 25
13 CAPTURE_SECONDS = 10
14 DATASET_DIR    = "dataset"
15 WEIGHTS_FILE   = "snn_weights.bin"
16 TRAIN_OUT_DIR  = "snn_training_output"
17 POOL_TYPE      = "avg"
18
19 # --- SNN / Biological neuron settings ---
20 T_STEPS        = 12          # temporal steps per image (simulation depth)
21 DT             = 1.0         # ms per time step
22 TAU_MEM        = 20.0        # membrane time constant (ms)
23 TAU_SYN        = 5.0         # synaptic time constant (ms)
24 V_TH          = 1.0         # base firing threshold
25 V_RESET       = 0.0         # reset potential after spike
26 MAX_INPUT_RATE = 0.8         # max Poisson firing probability per step (0-1)
27 SURROGATE_BETA = 1.0         # steepness of surrogate gradient
28
29 WEIGHT_CLIP    = 1.5
30 WEIGHT_DECAY   = 1e-4
31 SPARSITY_LAMBDA = 8e-4
32 TARGET_RATE    = 0.08        # target firing rate (fraction of T_STEPS)
33 HOMEOSTATIC_LR = 0.005
34 USE_DALES      = True
35 GRAD_CLIP      = 1.0
36 # =====

```

These tests use training data captured in the room at the start of the test, by sample audio and video for 10 seconds and thus do not share the same training data, however this is to my benefit as I can see many variations quickly.

The image based BIO SNN functions as follows with the kernel, image size and filter count being primary ways of adjusting the bottleneck which provides the most immediate results.

Spiking Autoencoder with LIF encoder and linear rate decoder.

Encoder:

Poisson(input pixels) -> Conv2D synaptic current -> LIF neurons

-> accumulate spikes over T_STEPS -> rate map -> pool -> bottleneck

Decoder:

Upsample bottleneck rates -> Conv2D -> reconstruction

(Linear decoder for stable gradient flow to the spiking bottleneck.)

Learning:

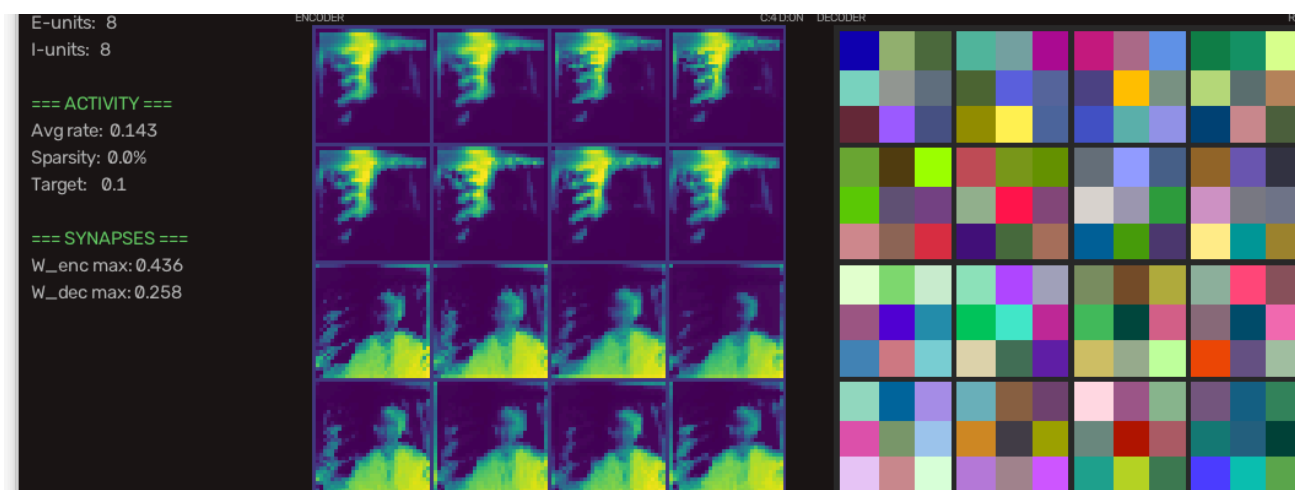
Single-step surrogate gradient through the LIF threshold.

Homeostatic bias + adaptive threshold plasticity.

Dale's principle + weight decay + sparsity.

I have also experimented with protected neurons with a simple toggle to block decay, however this is likely to show more effect with larger networks as a crude method to create “Protected Networks” but may not be needed if a network is large enough and the strongest connections have already formed, pruning will occur on the periphery of the network where the neurons are fresh and less reinforced, however it may be a trade off and make un-training difficult if early was initially poor. Additionally, biologically inspired networks seem to be far more chaotic and prone to decay and need to be fine tuned to prevent the network cascading towards a heavily biased or “dead” state.

Finally I have tested a basic neuron visualiser, however it needs further work:

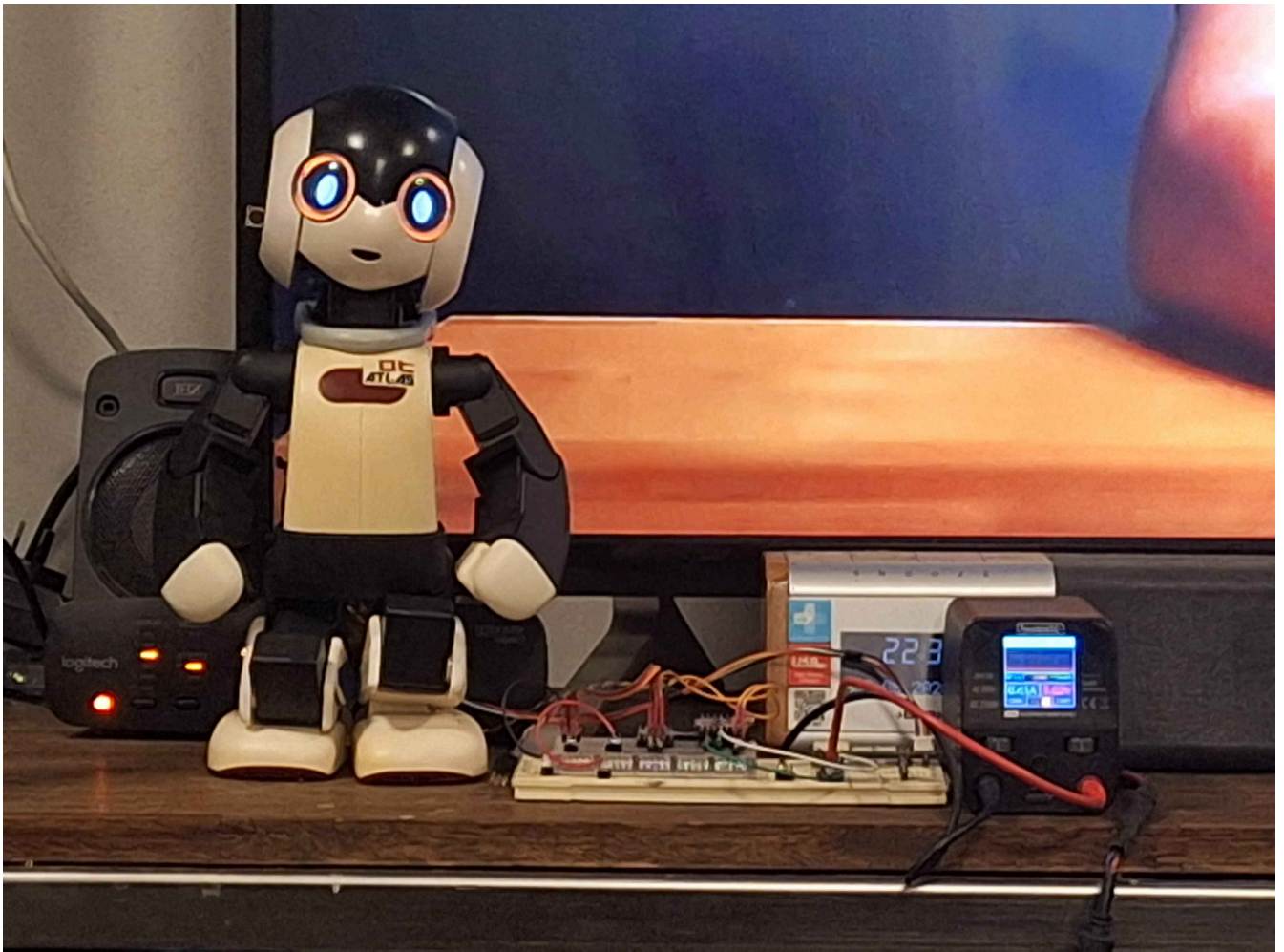


CURRENT PROJECT STATE

As it stands, efforts are now being focused on the new microcontroller that is vital to making the bot self contained and passes IMU data to the orangepi as well as interfaces with a mini amp to provide audio. Currently I am developing with a larger breadboard setup with some custom made circuit boards. The next step is hardware verification and finalisation of the schematics. After this point the new board can be manufactured and focus will primarily remain on small hardware fixes and building out the software since at this stage the robot will be hardware complete



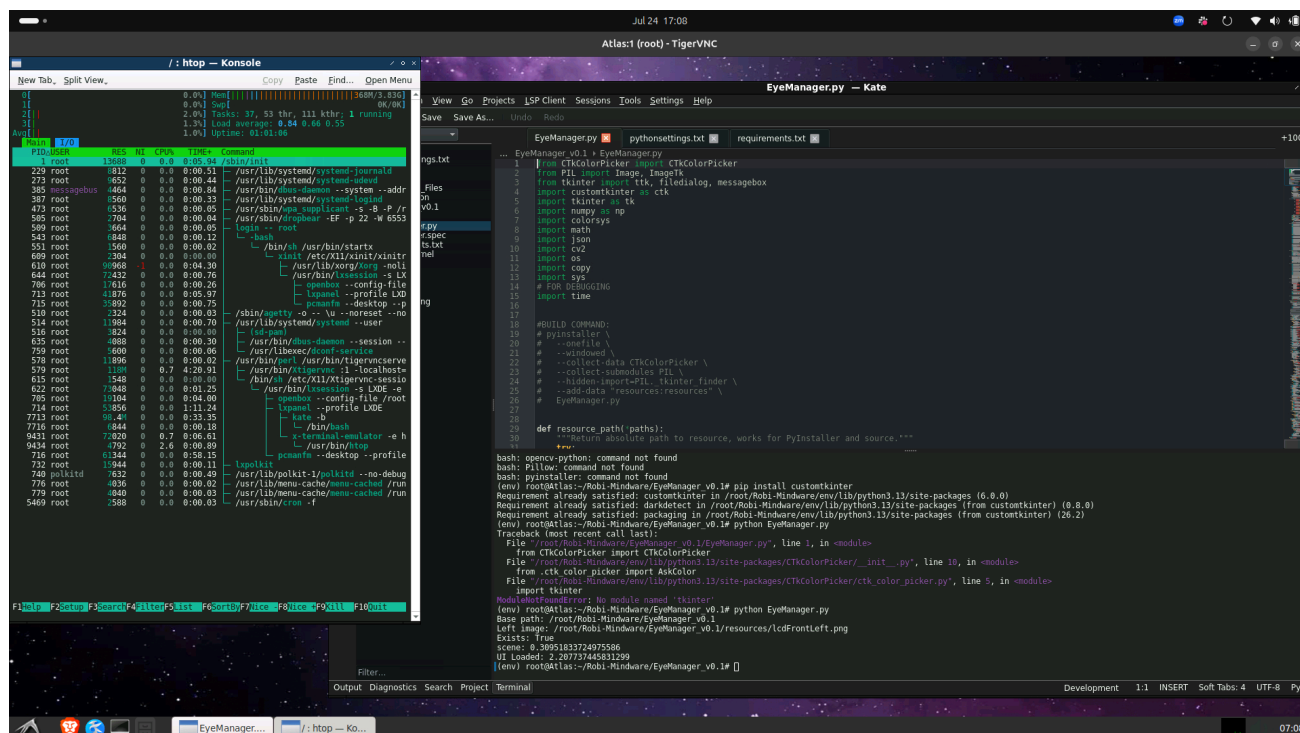
Testing LCD Breakout revision B, Corrected connector pin



Tying in systems to verify unified hardware setup and communications, no more laptop connections to test each sub system separately



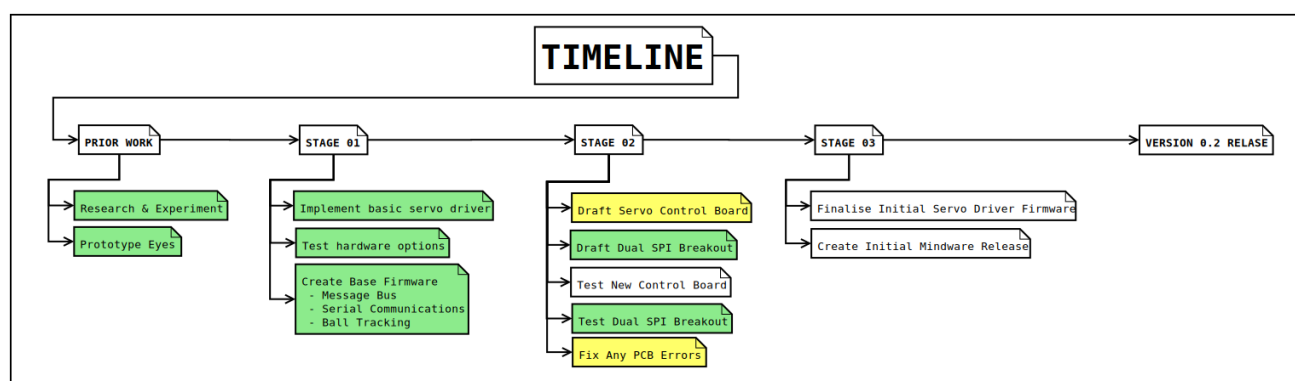
I have tested adding an additional motor to each arm to enable a basic grab function, this would do better with a more refined mechanism. This is currently done with cheap PWM servos just as a proof of concept.



Development is undertaken via Tiger VNC and SSH on DietPI. Allowing for convenient workspace for the software development side. Serial communications can be used to modify microcontroller properties to reduce the need for regular hardware access.

TIMELINE:

Hardware finalization and prototype software to be completed by end of year (2026) with multiple units then being constructed for testing machine-to-machine and machine-to-people interactions.



Main hardware focused milestones. At a crucial stage that integrates all main hardware.

PROPOSED EXPERIMENTS:

Social learning between 2 robots learning a queue/signal and then teaching it to the other bot without intervention.

POSSIBLE FUTURE AVENUES:

One of the long-term future avenues for social robots I see come from my experience seeing the people I cared for being unable to care for themselves and being resigned to a nursing home.

In the use in nursing homes for elderly patients who are often bed ridden or lonely. A social robot that can engage in a meaningful manner and learn from them like a friend does, this can help keep their minds active, and may help alleviate loneliness in these places and facilitate better communication with families.

“Social robot interventions had significant positive effects on decreasing depression and loneliness with large effect sizes. Group-based robot activities had a better effect on improving depression than individual-based robot activities. Longer durations of interventions produced significantly more improvement in depression.” (Yen et al., 2024).

But a question also arises when these people pass on, what happens to the robot? As we have seen recently, there have been attempts to make digital recreations of people and their likeness examples include celebrities, but even in court rooms where victims have been recreated with AI to testify, however this often comes off as disrespectful or even gross as can be seen in reporting media (Metzger & Kirchengast, 2025).

However in the case of a robot who has learned like a friend, they are not mimics or recreations of that person, but are artifacts and family heirlooms that facilitate the continuation of the knowledge that that person existed and to facilitate the passing of stories and lessons they wanted to share to the generations that followed them. The robot ultimately becomes a historian or “story teller” for the individual that imprinted on it.

REFERENCES

- De Agostini UK Ltd. (2015). *Build your own Robi: Pack 3: Stages 7–10—See Robi's head move for the first time* [Assembly guide].
De Agostini Publishing USA Inc. [Link to the Manual](#)
- Futaba Corporation. (2011). RS303MR/RS304MD Command Type Servo for Robot Instruction Manual [Instruction Manual/User Guide]
Futaba Corporation. [Link to the Manual](#)
- Metzger, J. & Kirchengast, T. (2025, June 19). Why a US court allowed a dead man to deliver his own victim impact statement – via an AI avatar. *UNSW Sydney Newsroom* [Article]
UNSW Sydney [Link to the Article](#)
- Yen, H.-Y., Huang, C.-W., Chiu, H.-L., & Jin, G. (2024). The effect of social robots on depression and loneliness for older residents in long-term care facilities: A meta-analysis of randomized controlled trials. *Journal of the American Medical Directors Association*, 25(6), 104979. <https://doi.org/10.1016/j.jamda.2024.02.017>

APPENDICES

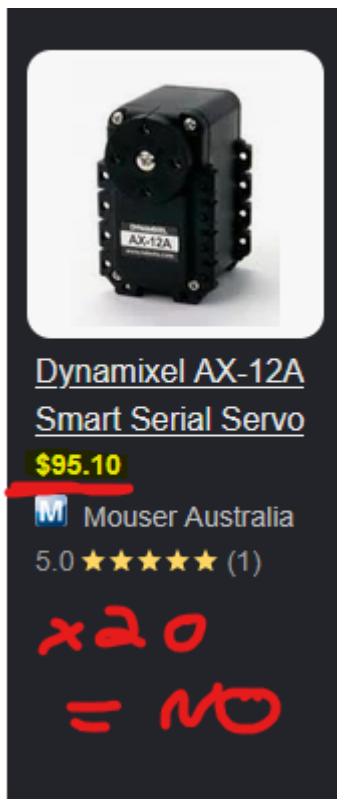


Fig 1.

<https://journals.plos.org/ploscompbiol/>